

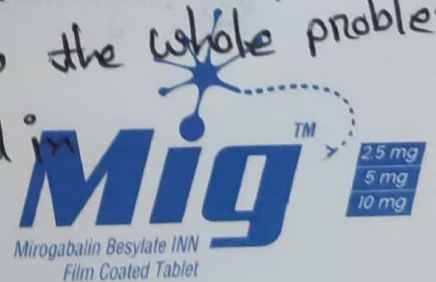
Greedy Method

Greedy Algorithm:

- ⇒ It is an algorithm that finds a solution to a problem in the shortest possible time.
- ⇒ It involves making the locally optimal choice at each stage with hope of finding the global optimum.
- Greedy algorithm always makes the choice that looks best at the moment.
- ⇒ Greedy algorithms constructs a solution to an optimization problem piece by piece through a sequence of choices that are:
 - Feasible
 - Locally optimal
 - Irrevocable (binding and abinding)
- ⇒ Optimal Solution $\left[\begin{array}{l} \text{Min cost} \\ \text{Max profit} \\ \text{Max risk} \end{array} \right.$

● Greedy Strategy:

- start with a solution to a small subproblem
- Build up to a solution to the whole problem
- Make choices that look good in short terms.



⇒ Advantages:

- Greedy algorithm work faster.
- Simple algorithm
- Easy to implement and understand.

⇒ Disadvantages:

- Greedy algorithm don't always work.
- Hard to prove correct.

• Greedy algorithm characteristics:

- Top-down approach
- Local optimality
- Not Backtracking
- Not allows the best result.

• Applications:

- ① Knapsack problem
- ② Minimum spanning tree
- ③ Activity selection problem
- ④ Huffman coding
- ⑤ Dijkstra's algorithm.

Differences between Greedy Techniques vs Dynamic Programming

Dynamic Programming	Greedy Techniques
• At each step, the choice is determined based on solutions of subproblems.	• At each step, we quickly make a choice that currently looks best.
• Sub-problems are solved first	• Greedy choice can be made first before solving further sub-problems
• Always gives a optimal solution.	• Could give a optimal solution or not.
• Bottom-up or top-down approach	• Top-down approach
• Can be slower	• Usually faster.
• More complex	• Simpler
• efficient	• Less efficient



Activity Selection Problem:

⇒ Select maximum number of activities to solve by a single person.

→ The problem is to select a maximum-size set of mutually compatible activities.

Example:

	a_1	a_2	a_3	a_4	a_5	a_6	a_7
s_i	2	1	4	1	9	8	5
f_i	5	8	7	3	11	10	9

- ① Sorting the finishing time
- ② find compatible activity and add to solution.

Step 1:

Sorting the finishing time

	a_4	a_1	a_3	a_2	a_7	a_6	a_5
s_i	1	2	4	1	5	8	9
f_i	3	5	7	8	9	10	11

$$A = \{a_4, a_3, a_6\}$$

Solution.

[Finishing the first
sort karta hain kya
kya possible hain
solution array
mein hai]

• Pseudocode :

Greedy-activity (s, f)

{

$n \leftarrow \text{length}[S]$

$A \leftarrow \{a_1\}$

$j \leftarrow \{1\}$

for $i \leftarrow 2$ to n

do if $s_i \geq f_i$

then $A \leftarrow A \cup \{i\}$

$j \leftarrow i$

return A ;

}

Time complexity :

$O(n \log n) + O(n)$

$\rightarrow O(n \log n)$

Knapsack problem:

- We are given 'n' objects and a knapsack. Object 'i' has a weight w_i and the knapsack has a capacity 'M'.
- If a fraction x_i , $0 \leq x_i \leq 1$, of object i is placed into the knapsack then a profit of $p_i x_i$ is earned.
- The objective is to obtain a filling of the knapsack that maximizes the total weight of all chosen objects to be at most M.

Example:

(Fraction knapsack)

Object	1	2	3
Profit	25	24	15
Weight	18	15	10

প্রথমে P/w এর ক্রম হবে। মনে পড়বে P/w বেশি প্রথম থেকে
Count করতে হবে, এর পর decreasing way এ গিয়ে
M fill করতে হবে।

Capacity (M) = 20

Solution:

P/w for objects, $ob_1 = \frac{25}{18} = 1.3$

$ob_2 = \frac{24}{15} = 1.6$

$ob_3 = \frac{15}{10} = 1.5$

(capacity count not weight but result show 25 Profit)

$$\therefore 24 + \frac{5}{10} \times 15 = 31.5$$

Knapsack 0/1 cannot be solved by greedy strategy

→ Let a knapsack's capacity is 12g. Three object given

	01	02	03
P	20	25	60
w	2	4	8
P/w	10	6.2	7.5

	→ 2
Ob 3	- 8
Ob 1	- 2

As P/w we will take ob1 and object 3 as their P/w is greater. Then there are still 2gm free space. But if we take 03 and 02 then it will fill up the space perfectly. So, that's why greedy method cannot solve 0/1 knapsack.

⇒ According to dynamic Programming, we can write the object in 2^n form. $n = \text{object number}$.

Here, $2^n = 2^3 = 8$

Now

ob1	ob2	ob3
0	0	0

0	0	1
---	---	---

Profit = 60

0	1	0
---	---	---

Profit = 25

0	1	1
---	---	---

" = $(60 + 25) = 85$

1	0	0
---	---	---

" = (20)

1	0	1
---	---	---

" = $(20 + 60) = 80$

1	1	0
---	---	---

" = $(20 + 25) = 45$

1	1	1
---	---	---

" = (105) X

000 and 111 is not possible

DP checks every possibility so Here we can apply DP.

Total profit for ob1 and ob3
is fill up space.



• Pseudo code:

for ($i=1$ to n)
 calculate P/w } $O(n)$

Sort objects in decreasing order
 of P/w ratio } $O(n \log n)$

for $i=1$ to n
 if ($M > 0$ and $w_i \leq M$)
 $M = M - w_i$
 $P = P + P_i$
 if ($M > 0$)
 $P = P + P_i \times \left(\frac{M}{w_i}\right)$ } $O(n)$

$\therefore$ Time complexity : $O(n) + O(n \log n) + O(n)$
 $= O(n \log n)$

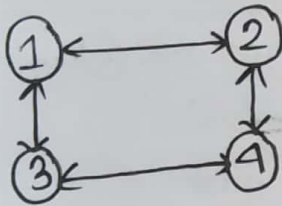
The Travelling Salesman Problem (TSP)

→ The travelling salesman problem consists of a salesman and a set of cities.

Has to visit each one of the cities starting from a certain one and returning to the same city.

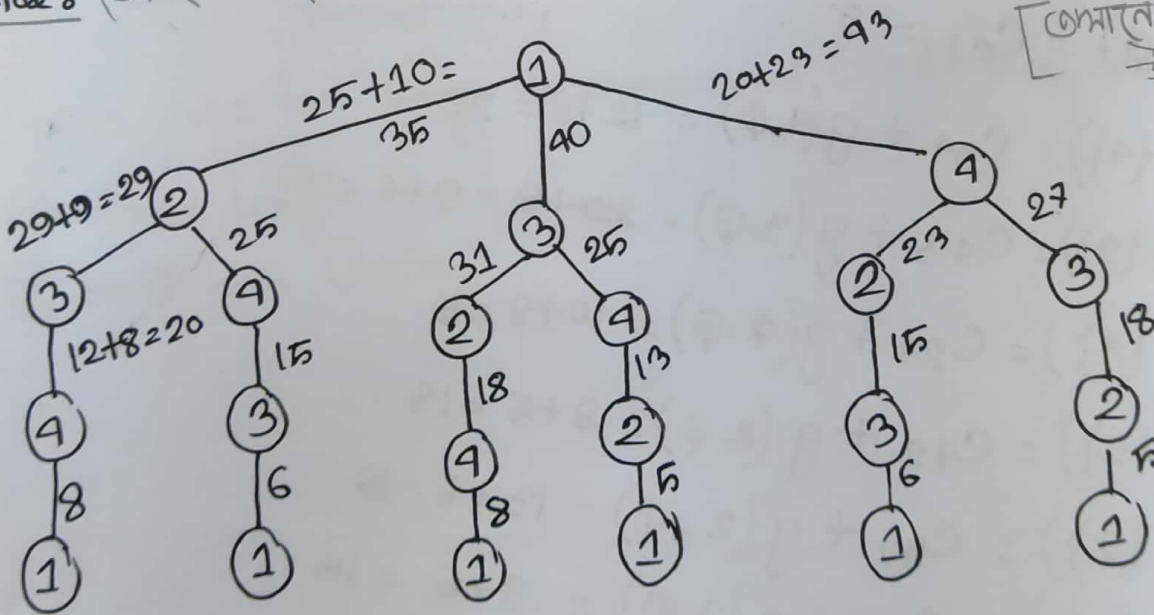
Finding the minimize length of the trip.

Example:



	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Tree: (আঁকতে ৩ পাশে না ৩ আঁকতে পারি)



[আমরা নিজেই cal. করেছি now]

The min cost is 35.

So the best path to choose is $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$

[৩য় শর্টকাট]



■ Formulae:

$$g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S - \{j\})\}$$

$g(i, S)$ is the shortest path starting from i and going through all vertex in S and terminal at 1.

Here, i = starting index

j = every node from i to terminal point.

S = Sum of node.

④ Can start from any point:

$$g(1, \emptyset) = c_{11} = 8$$

$$g(3, \emptyset) = c_{31} = 6$$

$$g(2, \emptyset) = c_{21} = 5$$

$$g(3, \{1\}) = c_{31} + g(1, \emptyset) = 12 + 8 = 20$$

$$g(4, \{1\}) = c_{41} + g(1, \emptyset) = 10 + 8 = 18$$

$$g(2, \{1\}) = c_{21} + g(1, \emptyset) = 8 + 5 = 13$$

$$g(4, \{2\}) = c_{42} + g(2, \emptyset) = 13 + 5 = 18$$

$$g(3, \{2\}) = c_{32} + g(2, \emptyset) = 9 + 6 = 15$$

$$g(2, \{3\}) = c_{23} + g(3, \emptyset) = 9 + 6 = 15$$

$$g(2, \{3, 1\}) = \begin{cases} c_{23} + g(3, \{1\}) = 9 + 20 = 29 \\ c_{21} + g(1, \{3\}) = 10 + 5 = 15 \end{cases}$$

$$g(3, \{2, 1\}) = \begin{cases} c_{32} + g(2, \{1\}) = 13 + 18 = 31 \\ c_{31} + g(1, \{2\}) = 14 + 13 = 27 \end{cases}$$

$$g(4, \{3, 2\}) = \begin{cases} c_{43} + g(3, \{2\}) = 27 \\ c_{42} + g(2, \{3\}) = 23 \end{cases}$$

$$g(1, \{2, 3, 4\}) = \begin{cases} c_{12} + g(2, \{3, 4\}) = 10 + 25 = 35 \\ c_{13} + g(3, \{2, 4\}) = 15 + 25 = 40 \\ c_{14} + g(4, \{2, 3\}) = 20 + 23 = 43 \end{cases}$$

So the ~~aboute~~ best path is $g(1, \{2, 3, 4\})$.

Huffman Codes:

- (i) For compressing data (sequence of characters)
- (ii) Widely used
- (iii) Very efficient (saving 20-90%)
- (iv) Use a table to keep frequencies of occurrence of characters.

There are two ways to compressed word. They are:

- ① Fixed length.
- ② Huffman Code.



Example 0

	A	B	C	D	E	F
Frequency	45	13	12	16	9	6

(Frequency मिला ना आकलन count कल्ल निले सल)

• Method 1

Fixed length = 2^n , n = bit number

$2^1 = 2$; it represent 2 letter

$2^2 = 4$; it represents 4 letter

$2^3 = 8$; it represents 8 letter

So 6 (A, B, C, D, E, F) letter length can be represented by 3 bit.

$$\therefore \text{Total bit} = (45 + 13 + 12 + 16 + 9 + 6) \times 3 \\ = 100 \times 3 = 300$$

	A	B	C	D	E	F
Frequency	45	13	12	16	9	6
Fixed length	000	001	010	011	100	101

(0-5) count कल्ल fixed length लललल सललल.

• Method 2%

A: 45 B: 13 C: 12 D: 16 E: 9 F: 5

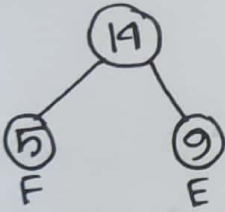
- ① element निम्न करना
- ② Sort करना
- ③ Tree छोटी करना, Parent शेषा छोटी element को लगाना

Sorting,

F: 5 E: 9 C: 12 B: 13 D: 16 A: 45

Now,

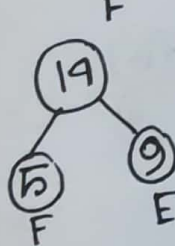
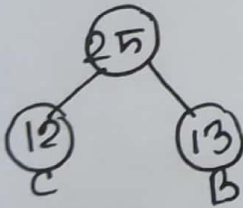
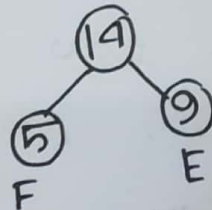
C: 12 B: 13 D: 16 A: 45



Again,

C: 12 B: 13

D: 16 A: 45

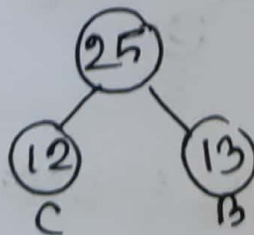
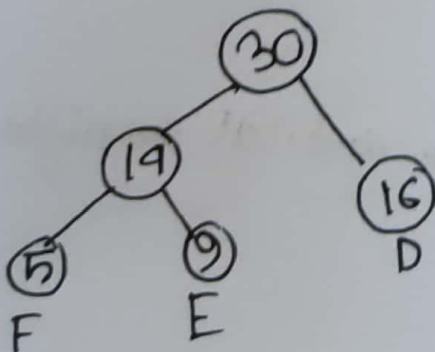
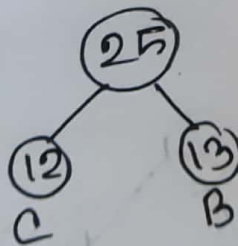
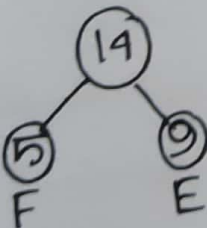


D: 16 A: 45

Again sorting,

D: 16

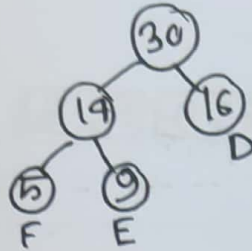
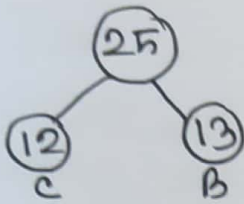
A: 45



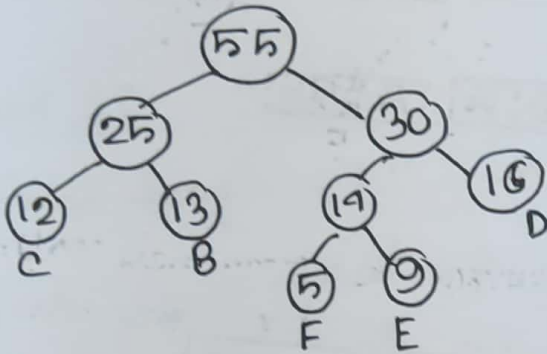
A: 45



Then,



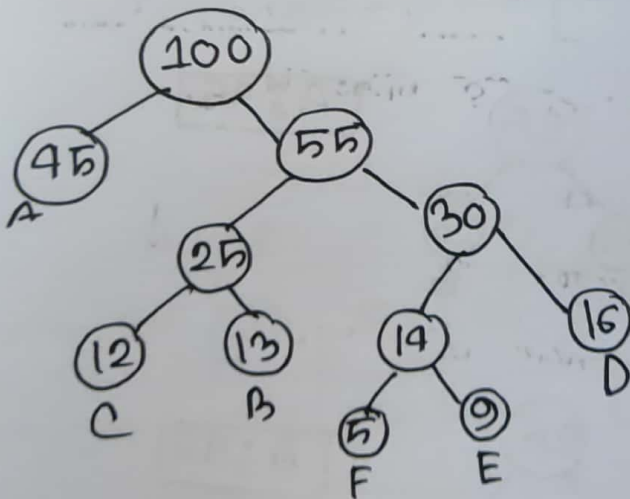
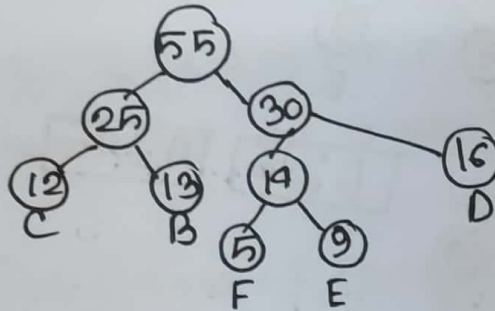
A: 45



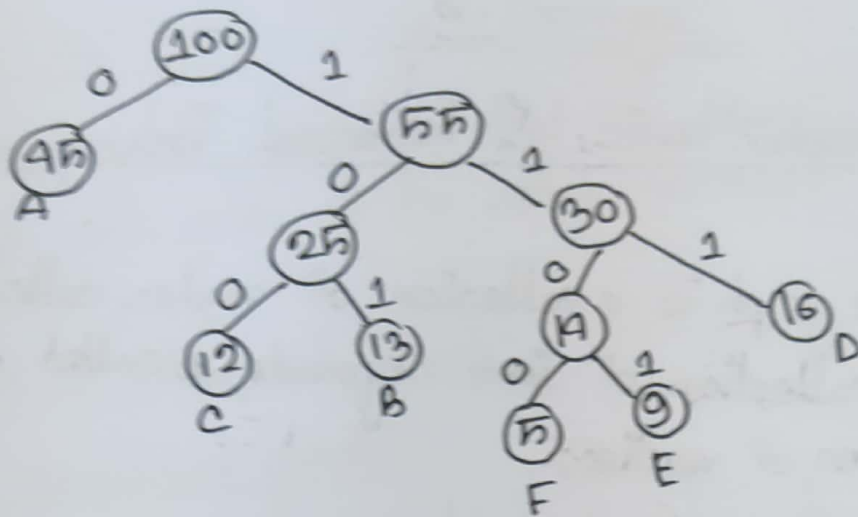
A: 45

Again Sorting,

A: 45



Now assigning 0 to every left node and 1 to right node.



Now,
 A = 0
 B = 101
 C = 100
 D = 111
 E = 1101
 F = 1100

मन्त्राण वरु frequency better ल छोटे bit दिस जाव छोटे frequency ल वरु bit दिस represent गरा शा.

$$\text{Total Bit} := 45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4 = 224 \text{ bit}$$

	A	B	C	D	E	F
Frequency	45	13	12	16	9	5
Huffman code	0	101	100	111	1101	1100

For Actual bit: (जस्त Reciever ल भण्डा वमल)

~~For Reciever = Total size + total number~~

For Reciever = Total size + total character X actual number of bits + total number of bits

$$\text{Fixed length} = 300 + 6 \times 8 + 18 = 366$$

$$\text{Huffman codes} = 224 + 6 \times 8 + 18 = 290$$

